NAMA: REYHAN GILANG RAMADHAN

KELAS: SI-IIIA

Soal 1 — Konsep Polymorphism

Jawaban:

Polymorphism adalah salah satu konsep dalam OOP yang memungkinkan method dengan nama yang sama memiliki perilaku atau hasil yang berbeda pada setiap class.

Contohnya pada sistem informasi perpustakaan, terdapat
method `tampilkan_peran()` yang digunakan oleh class Mahasiswa dan Dosen.

- Pada class Mahasiswa, method tersebut menampilkan peran sebagai mahasiswa.
- Pada class Dosen, method tersebut menampilkan peran sebagai dosen.

Contoh kode:

```
class Mahasiswa:    def tampilkan_peran(self):        print("Saya adalah Mahasiswa")class Dosen:    def tampilkan_peran(self):        print("Saya adalah Dosen")
```

Jadi, polymorphism membuat satu nama method dapat digunakan untuk menjalankan tindakan yang berbeda sesuai dengan object yang memanggilnya.

Soal 2 — Overriding dan Polymorphism

Jawaban:

Overriding adalah proses ketika subclass membuat ulang method yang sudah dimiliki oleh superclass dengan nama yang sama, tetapi memiliki isi atau hasil yang berbeda.

Sedangkan polymorphism adalah kemampuan suatu method untuk menghasilkan perilaku yang berbeda tergantung object yang menggunakannya.

Overriding menjadi dasar perilaku polimorfik pada Python karena setiap subclass dapat memiliki implementasi method sendiri. Ketika method dipanggil, Python akan menjalankan method sesuai dengan jenis object yang digunakan.

Dengan demikian, overriding memungkinkan polymorphism terjadi karena method yang sama dapat memberikan hasil yang berbeda.

Soal 3 — Prediksi Output

Jawaban:

Output yang muncul ketika program dijalankan adalah:

```
Mahasiswa
Dosen
```

Penjelasan:

Output tersebut muncul karena class Mahasiswa dan Dosen memiliki method `tampilkan_peran()` dengan isi yang berbeda.

Pada perulangan `for`, program memanggil method `tampilkan_peran()` dari setiap object yang ada di dalam daftar.

- Object Mahasiswa menampilkan tulisan Mahasiswa.
- Object Dosen menampilkan tulisan Dosen.

Konsep OOP yang ditunjukkan adalah Polymorphism, karena method yang memiliki nama sama dapat menghasilkan output berbeda sesuai dengan object yang digunakan.

Soal 4 — Duck Typing

Jawaban:

Fungsi `tampilkan()` dapat menerima kedua object karena class Mahasiswa dan Dosen sama-sama memiliki method `ringkasan()`.

Dalam Python, konsep Duck Typing memungkinkan sebuah object digunakan berdasarkan kemampuan atau method yang dimilikinya, bukan berdasarkan hubungan inheritance dengan class lain.

Contohnya:

- Object Mahasiswa memiliki method `ringkasan()` yang menghasilkan tulisan Mahasiswa — Budi.
- Object Dosen memiliki method `ringkasan()` yang menghasilkan tulisan Dosen — Sari.

Ketika fungsi `tampilkan()` dipanggil, Python langsung menjalankan method `ringkasan()` dari masing-masing object.

Jadi, kedua object dapat digunakan dalam fungsi yang sama meskipun tidak memiliki hubungan inheritance.

Soal 5 — Merancang Fungsi Polimorfik

Jawaban:

Berikut fungsi `tampilkan_semua(daftar)` yang dapat menerima beberapa object berbeda dan memanggil method `ringkasan()` pada setiap object.

Kode Python:

```
class Mahasiswa:    def ringkasan(self):        return "Mahasiswa - Budi"class Dosen:    def ringkasan(self):        return "Dosen - Sari"def tampilkan_semua(daftar):    for data in daftar:        print(data.ringkasan())m1 = Mahasiswa()d1 = Dosen()tampilkan_semua([m1, d1])
```

Output:

```
Mahasiswa - Budi
Dosen - Sari
```

Penjelasan:

Fungsi `tampilkan_semua(daftar)` menggunakan perulangan `for` untuk membaca setiap object yang ada di dalam daftar.

Kemudian, program memanggil method `ringkasan()` pada setiap object tanpa perlu memeriksa jenis class terlebih dahulu.

Jika terdapat object yang tidak memiliki method `ringkasan()`, maka Python akan menghasilkan error berupa:

```
AttributeError
```

Error tersebut terjadi karena object tidak memiliki method yang ingin dipanggil.

Oleh karena itu, setiap object yang dimasukkan ke dalam fungsi tersebut harus memiliki method `ringkasan()` agar program dapat berjalan dengan baik.

Kesimpulan

Polymorphism merupakan konsep OOP yang memungkinkan beberapa object menggunakan method dengan nama yang sama tetapi menghasilkan perilaku berbeda. Konsep ini membuat program lebih fleksibel, mudah dikembangkan, dan tidak perlu membuat fungsi terpisah untuk setiap jenis object.